**SC4052 Cloud Computing Assignment 1 Question 6 - A new approach to space-based data centres**
**Quah YiKai Gary U2322511D**

# 1. Introduction

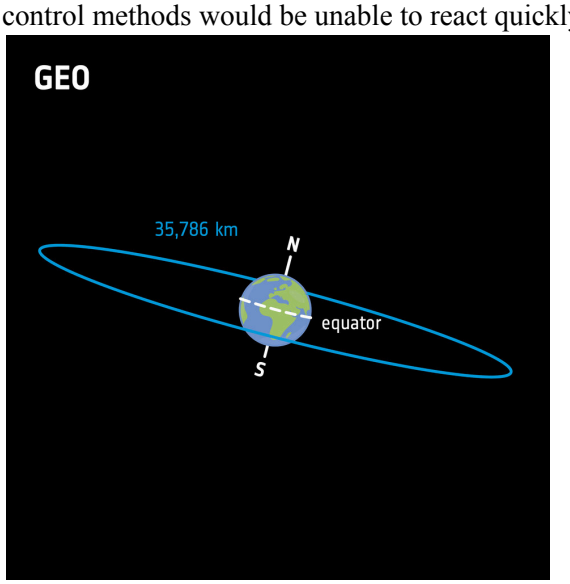
Satellites in space produce large amounts of data. Transmitting all unfiltered or unprocessed data would lead to a bottleneck in bandwidth. By processing the data in space prior to sending the processed data back to Earth through space-based data-centres, we can reduce overhead in overall bandwidth.

One major problem for space-based data centres would be unstable round-trip time (RTT) due to the data centre orbiting around the Earth, causing the distance between Earth and the data centre to fluctuate. The transmission rate is also further affected by other external factors, such as debris or other satellites.

# 2. Literature review

## 2.1 RTT in space

Earth has a geostationary (GEO) orbit of approximately 36, 000km [1], and due ot this distance, the approximated round-trip time (RTT) of a Geo satellite to Earth is 480ms [2], which has been noted to be much higher than typical transmission rates on Earth. With this large amount of RTT, network congestion control methods would be unable to react quickly to network congestion.



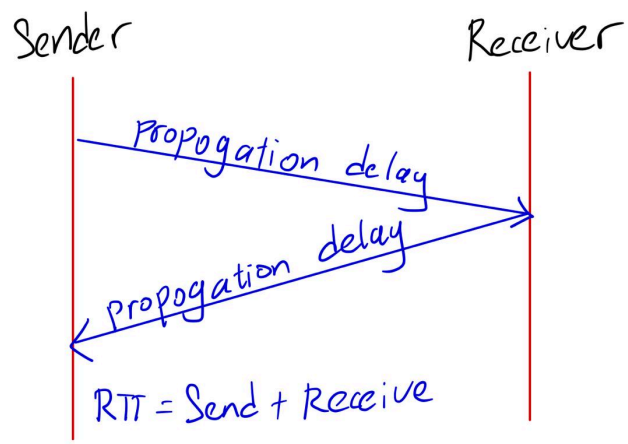




Figure 1: Geostationary Orbit [3]

However, it has been noted by studies that Low Earth Orbit (LEO) have potentially a lower RTT than networks on Earth. This is due to satellites transmitting data to Earth through radio waves travelling at 300,000km/s 500 - 1500 km from Earth [2] compared to data transmission on Earth through silica fibres at 200,000km/s [2]. Compared to satellites located in the GEO orbit of 36,000km, satellites in the LEO orbit potentially have a lower RTT than networks on Earth.

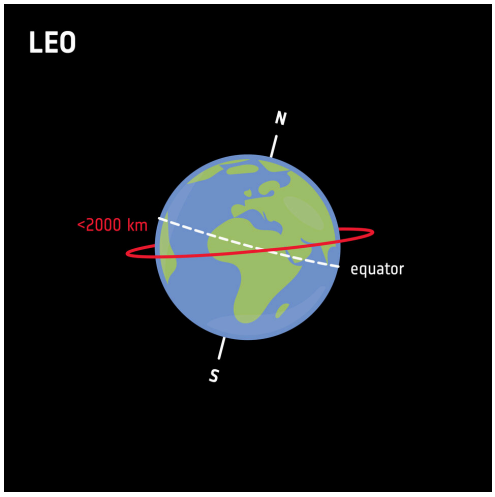


Figure 2: Low Earth orbit [3]

Although data centres in the LEO orbit might have faster transmission than on Earth, their RTT would fluctuate, causing a major technical challenge for space-based data centres, which would be a dynamically changing amount of RTT in space. Due to space-based data centres orbiting around the Earth, the distance between space-based data centres and Earth would vary, causing RTT to fluctuate. This is furthermore affected by the distance a space-based data centre would be positioned in Earth's orbit. This could also be affected by other space debris or satellites interfering with the microwaves transmitted from the data centre to Earth. With this many factors affecting RTT, fluctuating RTT is a concern for network congestion detection.

### 2.2 Additive Increase, Multiplicative Decrease (AIMD)

TCP uses Additive Increase, Multiplicative Decrease (AIMD) congestion control, where, when packet loss occurs, TCP reduces its sending rate (Multiplicative decrease), usually by half. TCP then gradually increases its send rate linearly over time (Additive increase) until congestion is detected again through packet loss.

The key limitation of TCP is that it only detects congestion after packet loss occurs, detecting network congestion when it is already too late, as packet loss occurs only after the queues are already full, resulting in packet drops, higher latency and inefficient network utilisation.

## Proposal

To address the fluctuating RTT in space, which causes network congestion detection instability, we could introduce an RTT-adaptive congestion control algorithm where the algorithm reacts to changing amounts of RTT, creating a more stable network.

A good place to start would be modifying the AIMD algorithm, given that it is stable, fair and has a high throughput.

### Additive increase with RTT

By scaling the additive increase with RTT, let's modify the traditional formula from **cwnd = cwnd + 1** to become **cwnd = cwnd + 1 / RTT**. However, if the RTT is too low, 1 / RTT might be too aggressive, causing packet drops, and when the RTT is too high, it would be too conservative, causing underutilization. Thus, we can further modify the formula to use a constant for the algorithm to scale off of, such as **cwnd = cwnd + k / RTT**, where k is a scalable constant.

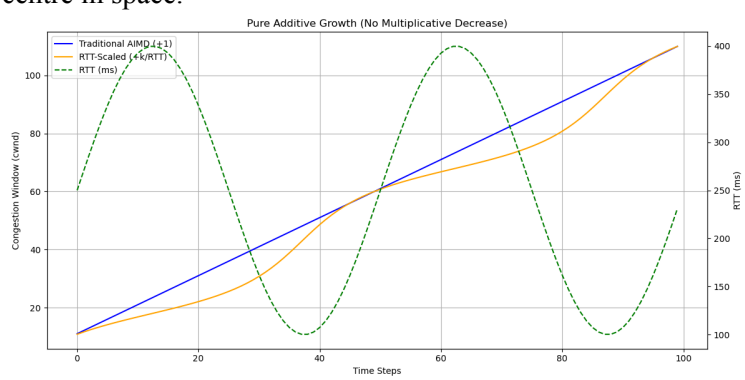
### Multiplicative decrease with RTT (cwnd = cwnd + k / RTT against cwnd = cwnd + 1)

For a typical multiplicative decrease, **cwnd = cwnd * 0.5.** As the RTT increases, we don't want to reduce the cwnd too aggressively, nor do we want to overshoot when RTT decreases. Thus, we could scale the cwnd with RTT through **cwnd = cwnd * 0.5 * ( RTTmin / RTTcurrent)**, where RTTmin is the optimal RTT from the data centre to Earth, calculated with the shortest distance, and RTTcurrent is the current estimated RTT from the data centre to Earth.

## Experimentation

### Additive increase

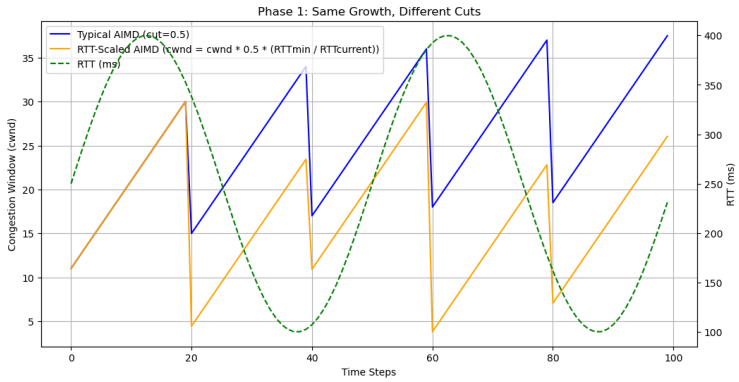
For this comparison, it is to prove that RTT-based AIMD reacts accordingly with the RTT of the data centre in space.



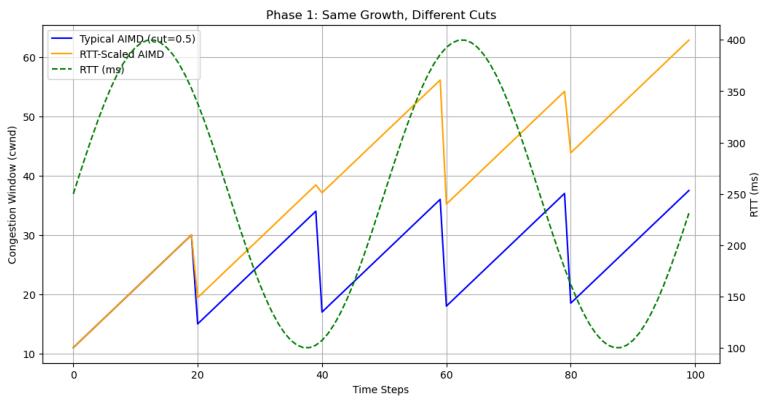

From the graph, we can note that as RTT decreases, the rate of cwnd increases. This is due to having a lower RTT, being able to provide faster congestion feedback in the network, allowing us to more aggressively probe the network. In contrast, when RTT is high, the rate of cwnd decreases, preventing cwnd from increasing too aggressively in case the algorithm overshoots capacity before detecting congestion. Although the algorithm might not be faster at certain points, it is noted to be safer and more stable in the long run.

### Multiplicative decrease ( cwnd = cwnd * 0.5 against cwnd = cwnd * 0.5 * (RTTmin / RTTcurrent)

For fair comparison, in the experimentation, we use the same growth rate, but different multiplicative decreases to demonstrate how the different multiplicative decrease affects the network congestion control.

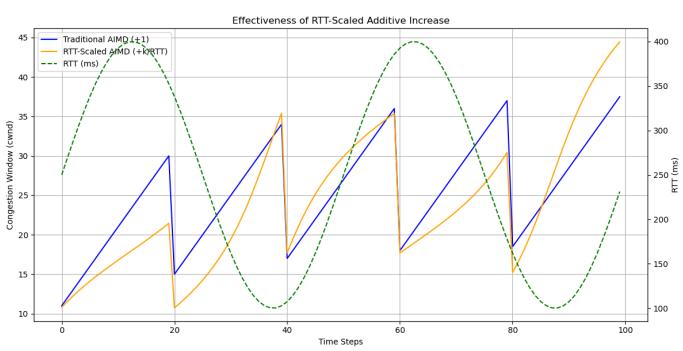



Through the same rate of growth, the cut for RTT-scaled AIMD is noted to be much more aggressive than a typical AIMD. A solution to this would be to cap the cut at AIMD's typical cut of 0.5.




Through capping the cut at a factor of 0.5, at certain points where RTT current is relatively small, RTT scaling AIMD is noted to have a less aggressive cut due to the scaling factor of RTTmin/RTTcurrent. Otherwise, in areas where RTT is large, it falls back to a factor of 0.5 to prevent aggressive cutting.

Although this max cap of 0.5 seems to have less aggressive cuts, we could possibly determine a better cap value rather than the standard 0.5.




# Conclusion

In summary, to tackle the issue of RTT instability causing network congestion control methods in space to be unstable, we can introduce RTT scaling into algorithms such as AIMD to make the algorithms react

accordingly to the varying RTTs. Although this method might not fully make use of the bandwidth, it would be more stable and reliable in the long run.

## Technical challenge and future outlook

Through modifying the formula further and with more experimentation and numerical analysis, RTT-scaled algorithms could be tuned to be better, such that the factor K (cwnd = cwnd + k / RTT) would have to be studied further to determine a better value for the proposed AIMD-RTT algorithm or the multiplicative decrease, the factor of 0.5 in (cwnd = cwnd * 0.5 * max(0.5, RTTmin/RTTcurrent)) would have to be studied further as well to determine a better clamp or multiplier for the algorithm.

Other factors or tuning can also be researched and experimented with, changing the formula entirely. All in all, RTT-adaptive congestion algorithms based on existing algorithms would definitely be beneficial for network congestion detection for space-borne data centres. More experimentation on other algorithms besides TCP AIMD could also be done to identify the best tuning parameters relative to RTT.

## Appendix

Code: https://github.com/GaryQuah/SC4052-Cloud-computing/blob/main/RTT-AIMD.ipynb

## References


1. The European Space Agency, “3. The geostationary orbit,” www.esa.int. https://www.esa.int/Education/3._The_geostationary_orbit
2. J. Conde, G. Martínez, P. Reviriego, and J. A. Hernández, “Round trip times (RTTs): comparing terrestrial and LEO satellite networks,” in 2024 27th Conference on Innovation in Clouds, Internet and Networks (ICIN), Paris, France: IEEE, Mar. 2024, pp. 42–46. doi: 10.1109/ICIN60470.2024.10494421.
3. The European Space Agency, “Types of orbits,” www.esa.int, Mar. 30, 2020. https://www.esa.int/Enabling_Support/Space_Transportation/Types_of_orbits#LEO